Pizza Runner — SQL Case Study #2

8 Week SQL Challenge by Danny Ma

Data Cleaning & Transformation

Before answering the case study questions, the data required significant cleaning. The following SQL statements were used to fix null representations, cast data types, normalize the pizza_recipes table, and add proper constraints.

Clean customer_orders

```
UPDATE customer_orders
SET exclusions = NULL
WHERE exclusions IN ('null', '');

UPDATE customer_orders
SET extras = NULL
WHERE extras IN ('null', '', 'NaN');
```

Clean runner_orders

```
UPDATE pizza_runner.runner_orders
SET pickup_time = NULL
WHERE pickup_time IN ('null', '');

ALTER TABLE pizza_runner.runner_orders
ALTER COLUMN pickup_time TYPE TIMESTAMP
USING pickup_time::TIMESTAMP;

UPDATE pizza_runner.runner_orders
SET distance = NULL
WHERE distance IN ('null', '');

ALTER TABLE pizza_runner.runner_orders
ALTER COLUMN distance TYPE NUMERIC(5,2)
USING substring(distance FROM '[0-9]+\.?[0-9]*')::NUMERIC;

UPDATE pizza_runner.runner_orders
SET duration = NULL
WHERE duration IN ('null', '');

UPDATE pizza_runner.runner_orders
SET duration = (substring(duration FROM '[0-9]+') || ' minutes')
WHERE duration IS NOT NULL;

ALTER TABLE pizza_runner.runner_orders
ALTER COLUMN duration TYPE INTERVAL
USING duration::INTERVAL;

UPDATE pizza_runner.runner_orders
SET cancellation = NULL
WHERE cancellation ILIKE ANY (ARRAY['null', '', 'NaN']);
```

Normalize pizza_recipes

```
CREATE TABLE order_toppings AS
SELECT
    pizza_id,
    regexp_split_to_table(toppings, ','):: int AS topping_id
FROM pizza_recipes;

DROP TABLE IF EXISTS pizza_recipes;
ALTER TABLE order_toppings RENAME TO pizza_recipes;
```

Add Primary Keys & Foreign Keys

```
ALTER TABLE pizza_toppings ADD PRIMARY KEY (topping_id);
ALTER TABLE pizza_names ADD PRIMARY KEY (pizza_id);
ALTER TABLE runners ADD PRIMARY KEY (runner_id);
ALTER TABLE runner_orders ADD PRIMARY KEY (order_id);

ALTER TABLE pizza_recipes
ADD CONSTRAINT fk_topping FOREIGN KEY (topping_id)
REFERENCES pizza_toppings(topping_id) ON DELETE CASCADE;

ALTER TABLE pizza_recipes
ADD CONSTRAINT fk_pizza FOREIGN KEY (pizza_id)
REFERENCES pizza_names(pizza_id) ON DELETE CASCADE;

ALTER TABLE customer_orders
ADD CONSTRAINT fk_pizza FOREIGN KEY (pizza_id)
REFERENCES pizza_names(pizza_id) ON DELETE CASCADE;

ALTER TABLE runner_orders
ADD CONSTRAINT fk_runner FOREIGN KEY (runner_id)
REFERENCES runners(runner_id) ON DELETE CASCADE;
```

Section A: Pizza Metrics

A1. How many pizzas were ordered?

SQL Query:

```
SELECT COUNT(*) AS total_unique_pizzas
FROM (
    SELECT DISTINCT order_id, pizza_id, order_time, exclusions, extras
    FROM customer_orders
) AS sub;
```

Output:

	total_unique_pizzas bigint
1	13

A2. How many unique customer orders were made?

SQL Query:

```
SELECT COUNT(DISTINCT customer_id)
FROM customer_orders;
```

Output:

	count bigint
1	5

A3. How many successful orders were delivered by each runner?

SQL Query:

```
SELECT DISTINCT runner_id,
COUNT(*) OVER (PARTITION BY runner_id) AS total_order_per_runner
FROM runner_orders
WHERE cancellation IS NULL;
```

Output:

	runner_id integer	total_order_per_runner bigint
1	2	3
2	1	4
3	3	1

A4. How many of each type of pizza was delivered?

SQL Query:

```
SELECT DISTINCT pizza_id, pizza_name,
COUNT(*) OVER (PARTITION BY pizza_id) AS total_pizza
FROM customer_orders
JOIN runner_orders USING (order_id)
JOIN pizza_names USING (pizza_id)
WHERE cancellation IS NULL;
```

Output:

	pizza_id integer	pizza_name text	total_pizza bigint
1	1	Meatlovers	9
2	2	Vegetarian	3

A5. How many Vegetarian and Meatlovers were ordered by each customer?

SQL Query:

```
SELECT customer_id, p.pizza_name, COUNT(pizza_id) AS ordered
FROM customer_orders c
JOIN pizza_names p USING (pizza_id)
GROUP BY customer_id, pizza_name
ORDER BY customer_id;
```

Output:

	customer_id integer	pizza_name text	ordered bigint
1	101	Meatlovers	2
2	101	Vegetarian	1
3	102	Meatlovers	2
4	102	Vegetarian	1
5	103	Meatlovers	3
6	103	Vegetarian	1
7	104	Meatlovers	3
8	105	Vegetarian	1

A6. What was the maximum number of pizzas delivered in a single order?

SQL Query:

```
WITH max_orders AS (
    SELECT order_id, COUNT(pizza_id) AS max_ordered
    FROM customer_orders
    GROUP BY order_id
)
SELECT MAX(max_ordered) AS max_ordered
FROM max_orders;
```

Output:

	max_ordered bigint
1	3

A7. For each customer, how many delivered pizzas had at least 1 change and how many had no changes?

SQL Query:

```
SELECT customer_id,
       COUNT(CASE WHEN exclusions IS NOT NULL OR extras IS NOT NULL THEN 1 END) AS
modified_pizzas,
       COUNT(CASE WHEN exclusions IS NULL AND extras IS NULL THEN 1 END) AS
unmodified_pizzas
FROM customer_orders
JOIN runner_orders USING (order_id)
WHERE cancellation IS NULL
GROUP BY customer_id
ORDER BY customer_id;
```

Output:

	customer_id integer	modified_pizzas bigint	unmodified_pizzas bigint
1	101	0	2
2	102	0	3
3	103	3	0
4	104	2	1
5	105	1	0

A8. How many pizzas were delivered that had both exclusions and extras?

SQL Query:

```
SELECT customer_id,
       COUNT(CASE WHEN exclusions IS NOT NULL AND extras IS NOT NULL THEN 1 END) AS
modified_pizzas
FROM customer_orders
JOIN runner_orders USING (order_id)
WHERE cancellation IS NULL
GROUP BY customer_id
ORDER BY customer_id;
```

Output:

	customer_id integer	modified_pizzas bigint
1	101	0
2	102	0
3	103	0
4	104	1
5	105	0

A9. What was the total volume of pizzas ordered for each hour of the day?

SQL Query:

```
SELECT DATE_PART('hour', order_time) AS intervals,
       COUNT(*) AS pizza_ordered
FROM customer_orders
GROUP BY intervals
ORDER BY intervals;
```

Output:

	intervals double precision	pizza_ordered bigint
1	11	1
2	13	3
3	18	3
4	19	1
5	21	3
6	23	3

A10. What was the volume of orders for each day of the week?

SQL Query:

```
SELECT EXTRACT(DOW FROM order_time) AS hour_of_day,
       COUNT(*) AS pizza_ordered
FROM customer_orders
```

```
GROUP BY hour_of_day
ORDER BY hour_of_day;
```

Output:

	hour_of_day numeric	pizza_ordered bigint
1	3	5
2	4	3
3	5	1
4	6	5

Section B: Runner and Customer Experience

B1. How many runners signed up for each 1 week period? (i.e. week starts 2021-01-01)

SQL Query:

```
SELECT DATE_TRUNC('week', registration_date) AS intervals,
       COUNT(DISTINCT runner_id) AS runner_signup
FROM runners
JOIN runner_orders USING (runner_id)
WHERE pickup_time IS NOT NULL
GROUP BY intervals
ORDER BY intervals;
```

Output:

	intervals timestamp with time zone	runner_signup bigint
1	2020-12-28 00:00:00+05:30	2
2	2021-01-04 00:00:00+05:30	1

B2. What was the average time in minutes it took for each runner to arrive at the Pizza Runner HQ to pickup the order?

SQL Query:

```
WITH orders AS (
    SELECT DISTINCT order_id, order_time
    FROM customer_orders
)
SELECT ROUND(
         AVG(EXTRACT(EPOCH FROM (r.pickup_time - o.order_time)) / 60),
         2
       ) AS avg_pickup_minutes
FROM runner_orders r
JOIN orders o USING (order_id)
WHERE r.pickup_time IS NOT NULL;
```

Output:

	avg_pickup_minutes numeric
1	15.98

B3. Is there any relationship between the number of pizzas and how long the order takes to prepare?

SQL Query:

```
WITH pizza_count AS (
    SELECT order_id, COUNT(pizza_id) AS num_pizzas
    FROM customer_orders
    GROUP BY order_id
),
prep_time AS (
    SELECT r.order_id,
           EXTRACT(EPOCH FROM (r.pickup_time - c.order_time)) / 60 AS prep_minutes
    FROM runner_orders r
    JOIN customer_orders c USING(order_id)
    WHERE r.pickup_time IS NOT NULL
)
SELECT p.num_pizzas, AVG(t.prep_minutes) AS avg_prep_time
FROM pizza_count p
JOIN prep_time t USING(order_id)
GROUP BY p.num_pizzas
ORDER BY p.num_pizzas;
```

Output:

	num_pizzas bigint	avg_prep_time numeric
1	1	12.3566666666666667
2	2	18.3750000000000000
3	3	29.2833333333333333

B4. What was the average distance travelled for each customer?

SQL Query:

```
SELECT order_id, customer_id, AVG(distance)
FROM runner_orders
JOIN customer_orders USING (order_id)
WHERE distance IS NOT NULL
GROUP BY customer_id, order_id
ORDER BY customer_id, order_id;
```

Output:

	order_id integer	customer_id integer	avg numeric
1	1	101	20.0000000000000000
2	2	101	20.0000000000000000
3	3	102	13.4000000000000000
4	8	102	23.4000000000000000
5	4	103	23.4000000000000000
6	5	104	10.0000000000000000
7	10	104	10.0000000000000000
8	7	105	25.0000000000000000

B5. What was the difference between the longest and shortest delivery times for all orders?

SQL Query:

```
SELECT MAX(duration) - MIN(duration) AS differnce
FROM runner_orders
WHERE duration IS NOT NULL;
```

Output:

	differnce interval
1	00:30:00

B6. What was the average speed for each runner for each delivery and do you notice any trend for these values?

SQL Query:

```
SELECT runner_id,
       order_id,
       ROUND(AVG(distance / (EXTRACT(EPOCH FROM duration) / 3600)), 2) AS prep_speed_kmh
FROM runner_orders
WHERE duration IS NOT NULL
```

```
GROUP BY runner_id, order_id
ORDER BY runner_id, order_id;
```

Output:

	runner_id integer	order_id [PK] integer	prep_speed_kmh numeric
1	1	1	37.50
2	1	2	44.44
3	1	3	40.20
4	1	10	60.00
5	2	4	35.10
6	2	7	60.00
7	2	8	93.60
8	3	5	40.00

B7. What is the successful delivery percentage for each runner?

SQL Query:

```
SELECT runner_id,
       COUNT(*) AS total_orders,
       COUNT(CASE WHEN cancellation IS NULL THEN 1 END) AS successful_orders,
       ROUND(
           100.0 * COUNT(CASE WHEN cancellation IS NULL THEN 1 END) / COUNT(*),
           2
       ) AS success_percentage
FROM runner_orders
GROUP BY runner_id
ORDER BY runner_id;
```

Output:

	runner_id integer	total_orders bigint	successful_orders bigint	success_percentage numeric
1	1	4	4	100.00
2	2	4	3	75.00
3	3	2	1	50.00

Section C: Ingredient Optimisation

C1. What are the standard ingredients for each pizza?

SQL Query:

```
SELECT pn.pizza_name,
       ARRAY_AGG(pt.topping_name ORDER BY pt.topping_name) AS standard_ingredients
FROM pizza_recipes pr
JOIN pizza_names pn USING (pizza_id)
JOIN pizza_toppings pt USING (topping_id)
GROUP BY pn.pizza_name
ORDER BY pn.pizza_name;
```

Output:

	pizza_name text	standard_ingredients text[]
1	Meatlovers	{Bacon,"BBQ Sauce",Beef,Cheese,Chicken,Mushrooms,Pepperoni,Sala...
2	Vegetarian	{Cheese,Mushrooms,Onions,Peppers,"Tomato Sauce",Tomatoes}

C2. What was the most commonly added extra?

SQL Query:

```
WITH extras_split AS (
    SELECT regexp_split_to_table(extras, ',')::int AS topping_id
    FROM customer_orders
    WHERE extras IS NOT NULL
)
SELECT pt.topping_name,
       COUNT(*) AS times_added
FROM extras_split
JOIN pizza_toppings pt USING(topping_id)
GROUP BY pt.topping_name
ORDER BY times_added DESC
LIMIT 1;
```

Output:

	topping_name text	times_added bigint
1	Bacon	4

C3. What was the most common exclusion?

SQL Query:

```
WITH extras_split AS (
    SELECT regexp_split_to_table(exclusions, ',')::int AS topping_id
    FROM customer_orders
    WHERE exclusions IS NOT NULL
)
SELECT pt.topping_name,
       COUNT(*) AS times_added
FROM extras_split
JOIN pizza_toppings pt USING(topping_id)
GROUP BY pt.topping_name
ORDER BY times_added DESC
LIMIT 1;
```

Output:

	topping_name text	times_added bigint
1	Cheese	4

C4. Generate an order item for each record in the customers_orders table in the format of one of the following:

Meat Lovers | Meat Lovers - Exclude Beef | Meat Lovers - Extra Bacon | Meat Lovers - Exclude Cheese, Bacon - Extra Mushroom, Peppers

SQL Query:

```
WITH order_cte AS (
    SELECT
        co.order_id,
        pn.pizza_name,
        (
            SELECT string_agg(pt.topping_name, ', ')
            FROM regexp_split_to_table(co.exclusions, ',') AS id
            JOIN pizza_toppings pt ON pt.topping_id = id::int
        ) AS exclusions,
        (
            SELECT string_agg(pt.topping_name, ', ')
            FROM regexp_split_to_table(co.extras, ',') AS id
            JOIN pizza_toppings pt ON pt.topping_id = id::int
        ) AS extras
    FROM customer_orders co
    JOIN pizza_names pn ON co.pizza_id = pn.pizza_id
)
SELECT
    order_id,
    pizza_name
    || CASE WHEN exclusions IS NOT NULL
            THEN ' - Exclude ' || exclusions ELSE '' END
    || CASE WHEN extras IS NOT NULL
            THEN ' - Extra ' || extras ELSE '' END
```

```
    AS order_item
FROM order_cte
ORDER BY order_id;
```

Output:

	order_id integer	order_item text
1	1	Meatlovers
2	2	Meatlovers
3	3	Meatlovers
4	3	Vegetarian
5	4	Meatlovers - Exclude Cheese
6	4	Meatlovers - Exclude Cheese
7	4	Vegetarian - Exclude Cheese
8	5	Meatlovers - Extra Bacon
9	6	Vegetarian
10	7	Vegetarian - Extra Bacon
11	8	Meatlovers
12	9	Meatlovers - Exclude Cheese - Extra Bacon, Chicken
13	10	Meatlovers - Exclude BBQ Sauce, Mushrooms - Extra Bacon, Che...
14	10	Meatlovers

C5. Generate an alphabetically ordered comma separated ingredient list for each pizza order from the customer_orders table and add a 2x in front of any relevant ingredients.

For example: "Meat Lovers: 2xBacon, Beef, ... , Salami"

SQL Query:

```
WITH
base_cte AS (
    SELECT co.order_id, pt.topping_name
    FROM customer_orders co
    JOIN pizza_recipes pr USING (pizza_id)
    JOIN pizza_toppings pt USING (topping_id)
),
excluded_cte AS (
    SELECT co.order_id, pt.topping_name
    FROM customer_orders co
    JOIN regexp_split_to_table(co.exclusions, ',') AS e(tid) ON TRUE
    JOIN pizza_toppings pt ON pt.topping_id = e.tid::int
    WHERE co.exclusions IS NOT NULL
),
extras_cte AS (
    SELECT co.order_id, pt.topping_name
    FROM customer_orders co
    JOIN regexp_split_to_table(co.extras, ',') AS e(tid) ON TRUE
    JOIN pizza_toppings pt ON pt.topping_id = e.tid::int
    WHERE co.extras IS NOT NULL
```

```
),
final_rows AS (
    SELECT order_id, topping_name FROM base_cte
    WHERE (order_id, topping_name) NOT IN (
        SELECT order_id, topping_name FROM excluded_cte
    )
    UNION ALL
    SELECT * FROM extras_cte
),
counted AS (
    SELECT order_id, topping_name, COUNT(*) AS cnt
    FROM final_rows
    GROUP BY order_id, topping_name
)
SELECT
    co.order_id,
    pn.pizza_name || ': ' ||
    STRING_AGG(
        CASE WHEN c.cnt = 2 THEN '2x' || c.topping_name
             ELSE c.topping_name END,
        ', ' ORDER BY c.topping_name
    ) AS final_ingredients
FROM counted c
JOIN customer_orders co USING (order_id)
JOIN pizza_names pn USING (pizza_id)
GROUP BY co.order_id, pn.pizza_name
ORDER BY co.order_id;
```

Output:

	order_id integer	final_ingredients text
1	1	Meatlovers: Bacon, BBQ Sauce, Beef, Cheese, Chicken, Mushrooms, Pepperoni, Salami
2	2	Meatlovers: Bacon, BBQ Sauce, Beef, Cheese, Chicken, Mushrooms, Pepperoni, Salami
3	3	Meatlovers: Bacon, BBQ Sauce, Beef, 2xCheese, Chicken, 2xMushrooms, Onions, Pepperoni, Peppers, Salami, Tomato Sauce, Tomatoes
4	3	Vegetarian: Bacon, BBQ Sauce, Beef, 2xCheese, Chicken, 2xMushrooms, Onions, Pepperoni, Peppers, Salami, Tomato Sauce, Tomatoes
5	4	Meatlovers: 2xBacon, 2xBacon, 2xBBQ Sauce, 2xBBQ Sauce, 2xBeef, 2xBeef, 2xChicken, 2xChicken, Mushrooms, Mushrooms, Onions, Oni
6	4	Vegetarian: 2xBacon, 2xBBQ Sauce, 2xBeef, 2xChicken, Mushrooms, Onions, 2xPepperoni, Peppers, 2xSalami, Tomato Sauce, Tomatoes
7	5	Meatlovers: 2xBacon, BBQ Sauce, Beef, Cheese, Chicken, Mushrooms, Pepperoni, Salami
8	6	Vegetarian: Cheese, Mushrooms, Onions, Peppers, Tomato Sauce, Tomatoes
9	7	Vegetarian: Bacon, Cheese, Mushrooms, Onions, Peppers, Tomato Sauce, Tomatoes
10	8	Meatlovers: Bacon, BBQ Sauce, Beef, Cheese, Chicken, Mushrooms, Pepperoni, Salami
11	9	Meatlovers: 2xBacon, BBQ Sauce, Beef, 2xChicken, Mushrooms, Pepperoni, Salami
12	10	Meatlovers: Bacon, Bacon, 2xBeef, 2xBeef, Cheese, Cheese, 2xChicken, 2xChicken, 2xPepperoni, 2xPepperoni, 2xSalami, 2xSalami

C6. What is the total quantity of each ingredient used in all delivered pizzas sorted by most frequent first?

SQL Query:

```
WITH base_cte AS (
    SELECT co.order_id, pt.topping_name
    FROM customer_orders co
    JOIN pizza_recipes pr USING (pizza_id)
    JOIN pizza_toppings pt USING (topping_id)
```

```
),
excluded_cte AS (
    SELECT co.order_id, pt.topping_name
    FROM customer_orders co
    JOIN regexp_split_to_table(co.exclusions, ',') AS e(tid) ON TRUE
    JOIN pizza_toppings pt ON pt.topping_id = e.tid::int
    WHERE co.exclusions IS NOT NULL
),
extras_cte AS (
    SELECT co.order_id, pt.topping_name
    FROM customer_orders co
    JOIN regexp_split_to_table(co.extras, ',') AS e(tid) ON TRUE
    JOIN pizza_toppings pt ON pt.topping_id = e.tid::int
    WHERE co.extras IS NOT NULL
),
final_rows AS (
    SELECT order_id, topping_name FROM base_cte
    WHERE (order_id, topping_name) NOT IN (
        SELECT order_id, topping_name FROM excluded_cte
    )
    UNION ALL
    SELECT * FROM extras_cte
)
SELECT topping_name, COUNT(*) AS total_quantity
FROM final_rows
GROUP BY topping_name
ORDER BY total_quantity DESC;
```

Output:

	topping_name text	total_quantity bigint
1	Bacon	14
2	Mushrooms	12
3	Chicken	11
4	Cheese	11
5	Pepperoni	10
6	Salami	10
7	Beef	10
8	BBQ Sauce	8
9	Tomatoes	4
10	Onions	4
11	Peppers	4
12	Tomato Sauce	4

Section D: Pricing and Ratings

D1. If a Meat Lovers pizza costs $12 and Vegetarian costs $10 and there were no charges for changes — how much money has Pizza Runner made so far if there are no delivery fees?

SQL Query:

```
WITH pizza_per_runner AS (
    SELECT runner_id,
           COUNT(CASE WHEN pizza_id = 1 THEN 1 END) AS meat_cnt,
           COUNT(CASE WHEN pizza_id = 2 THEN 1 END) AS veg_cnt
    FROM customer_orders
    JOIN runner_orders USING(order_id)
    WHERE cancellation IS NULL
    GROUP BY runner_id
)
SELECT runner_id,
       (meat_cnt * 12) + (veg_cnt * 10) AS revenue
FROM pizza_per_runner;
```

Output:

	runner_id integer	revenue bigint
1	1	70
2	2	56
3	3	12

D2. What if there was an additional $1 charge for any pizza extras?

SQL Query:

```
WITH pizza_per_runner AS (
    SELECT
        ro.runner_id,
        COUNT(CASE WHEN co.pizza_id = 1 THEN 1 END) AS meat_cnt,
        COUNT(CASE WHEN co.pizza_id = 2 THEN 1 END) AS veg_cnt,
        COUNT(CASE WHEN co.extras IS NOT NULL THEN 1 END) AS extra_cnt
    FROM customer_orders co
    JOIN runner_orders ro USING(order_id)
    WHERE ro.cancellation IS NULL
    GROUP BY ro.runner_id
```

```
)
SELECT runner_id,
       (meat_cnt * 12 + veg_cnt * 10 + extra_cnt * 1) AS revenue
FROM pizza_per_runner
ORDER BY revenue DESC;
```

Output:

	runner_id integer	revenue bigint
1	1	71
2	2	57
3	3	13

D3. The Pizza Runner team now wants to add an additional ratings system that allows customers to rate their runner — how would you design an additional table for this new dataset?

SQL Query:

```
CREATE TABLE pizza_runner.ratings (
    rating_id  SERIAL PRIMARY KEY,
    runner_id  INT NOT NULL,
    customer_id INT NOT NULL,
    order_id   INT NOT NULL,
    rating     SMALLINT CHECK (rating BETWEEN 1 AND 5),
    FOREIGN KEY (runner_id) REFERENCES runners(runner_id),
    FOREIGN KEY (order_id) REFERENCES runner_orders(order_id)
);

INSERT INTO pizza_runner.ratings (runner_id, customer_id, order_id, rating)
VALUES
    (1, 101, 1, 5),
    (1, 101, 2, 4),
    (1, 102, 3, 5),
    (2, 103, 4, 3),
    (3, 104, 5, 4),
    (2, 105, 7, 5),
    (2, 102, 8, 4),
    (1, 104, 10, 5);
```

(Schema creation & data insertion — no output table)

D4. Using your newly generated table — can you join all of the information together to form a table for successful deliveries? (customer_id, order_id, runner_id, rating, order_time, pickup_time, time between order and pickup, delivery duration, average speed, total number of pizzas)

SQL Query:

```
WITH successful_orders AS (
    SELECT order_id, runner_id, pickup_time, distance, duration
    FROM runner_orders
    WHERE cancellation IS NULL
),
pizza_count AS (
    SELECT order_id, COUNT(*) AS total_pizzas
    FROM customer_orders
    GROUP BY order_id
)
SELECT
    co.customer_id,
    co.order_id,
    so.runner_id,
    r.rating,
    co.order_time,
    so.pickup_time,
    ROUND(EXTRACT(EPOCH FROM (so.pickup_time - co.order_time)) / 60, 2)
        AS time_between_order_and_pickup,
    so.duration AS delivery_duration,
    ROUND(so.distance / (EXTRACT(EPOCH FROM so.duration) / 3600), 2)
        AS average_speed_kmh,
    pc.total_pizzas
FROM customer_orders co
JOIN successful_orders so USING (order_id)
JOIN pizza_count pc USING (order_id)
LEFT JOIN ratings r
       ON r.order_id = co.order_id
      AND r.runner_id = so.runner_id
ORDER BY co.order_id;
```

Output:

	customer_id integer	order_id integer	runner_id integer	rating smallint	order_time timestamp without time zone	pickup_time timestamp without time zone	time_between_order_and_pickup numeric	delivery_duration interval	average_speed_kmh numeric	total_pizzas bigint
1	101	1	1	5	2020-01-01 18:05:02	2020-01-01 18:15:34	10.53	00:32:00	37.50	1
2	101	2	1	4	2020-01-01 19:00:52	2020-01-01 19:10:54	10.03	00:27:00	44.44	1
3	102	3	1	5	2020-01-02 23:51:23	2020-01-03 00:12:37	21.23	00:20:00	40.20	2
4	102	3	1	5	2020-01-02 23:51:23	2020-01-03 00:12:37	21.23	00:20:00	40.20	2
5	103	4	2	3	2020-01-04 13:23:46	2020-01-04 13:53:03	29.28	00:40:00	35.10	3
6	103	4	2	3	2020-01-04 13:23:46	2020-01-04 13:53:03	29.28	00:40:00	35.10	3
7	103	4	2	3	2020-01-04 13:23:46	2020-01-04 13:53:03	29.28	00:40:00	35.10	3
8	104	5	3	4	2020-01-08 21:00:29	2020-01-08 21:10:57	10.47	00:15:00	40.00	1
9	105	7	2	5	2020-01-08 21:20:29	2020-01-08 21:30:45	10.27	00:25:00	60.00	1
10	102	8	2	4	2020-01-09 23:54:33	2020-01-10 00:15:02	20.48	00:15:00	93.60	1
11	104	10	1	5	2020-01-11 18:34:49	2020-01-11 18:50:20	15.52	00:10:00	60.00	2
12	104	10	1	5	2020-01-11 18:34:49	2020-01-11 18:50:20	15.52	00:10:00	60.00	2

D5. If a Meat Lovers pizza was $12 and Vegetarian $10 fixed prices with no cost for extras and each runner is paid $0.30 per kilometre traveled — how much money does Pizza Runner have left over after these deliveries?

SQL Query:

```
WITH total_earn AS (
    SELECT SUM(CASE WHEN pizza_id = 1 THEN 12 ELSE 10 END)::numeric AS earn
    FROM customer_orders co
    JOIN runner_orders ro ON co.order_id = ro.order_id
    WHERE ro.distance IS NOT NULL
),
total_spent AS (
    SELECT SUM(distance) * 0.30 AS spent
    FROM runner_orders
    WHERE distance IS NOT NULL
)
SELECT total_earn.earn - total_spent.spent AS profit
FROM total_earn
CROSS JOIN total_spent;
```

Output:

	profit numeric
1	94.4400